

[9] AnDB: Breaking Boundaries with an AI-Native Database for Universal Semantic Analysis

저자: Tao Wang, Xiang Xue, Guang Li, Yan Wang **학회/년도:** arXiv:2502.13805, 2025 **분량:** 약 8페이지 (데모 논문) **역할군:** (C) 경쟁/비교 시스템

요약

AnDB는 **AI 네이티브 데이터베이스**의 최신 동향을 보여주는 시스템으로, SQL 자체를 시맨틱 토큰으로 확장하여 비구조화 데이터에 대한 **선언적 시맨틱 분석**을 가능하게 한다.

핵심 아이디어는 전통적인 "SQL로 어떻게 벡터 검색을 표현할 것인가?"라는 문제에서 벗어나, "SQL 자체를 의미 기반 연산에 맞게 재설계하자"는 것이다. SQL에 3가지 시맨틱 토큰(PROMPT, SEM_MATCH, SEM_GROUP)을 추가하여, LLM 호출, 시맨틱 유사도 매칭, 시맨틱 그룹핑을 자연스럽게 표현할 수 있다.

핵심 기술: 1. **PROMPT 토큰:** SQL 내에서 LLM을 직접 호출 2. **SEM_MATCH 토큰:** 선언적 시맨틱 유사도 매칭 3. **SEM_GROUP 토큰:** 시맨틱 유사성 기반 그룹핑 4. **비용-정확도 옵티마이저:** LLM 호출 비용과 추론 정확도를 균형 맞춰 실행 계획 선택

본 논문과의 관계: Exqutor가 "기존 SQL에 벡터 검색을 추가"하는 보수적 접근이라면, AnDB는 "SQL 자체를 AI 연산에 맞게 재설계"하는 급진적 접근이다. AnDB 같은 시스템이 발전할수록, SEM_MATCH와 SEM_GROUP의 카디널리티를 정확히 추정해야 하므로, Exqutor 스타일의 카디널리티 추정이 **더욱 필요**해질 것이다.

상세분석

9.1 해결하고자 하는 문제

전통적 DB와 AI의 불일치

현대 데이터는 **구조화 데이터(Structured)**와 **비구조화 데이터(Unstructured)**의 혼합이다:

구조화 데이터 (90년대 중심): - 숫자: 매출, 수량, 나이 - 날짜: 주문 날짜, 생일 - 범주: 카테고리, 상태 - SQL로 정확히 분석 가능

비구조화 데이터 (현재의 90%): - 텍스트: 고객 리뷰, 제품 설명, 뉴스 - 이미지: 상품 사진, 사용자 사진 - 오디오: 고객 콜, 음성 피드백 - 비디오: 제품 데모, 사용 가영

근본적 문제:

문제: 비구조화 데이터의 "의미" 분석

전통 SQL:

```
SELECT * FROM reviews WHERE review_text LIKE '%좋음%'
```

→ 정확한 텍스트 일치만 가능

→ "좋아", "훌륭함", "최고" 등 유사한 표현을 못 찾음

필요한 분석:

```
SELECT * FROM reviews
```

```
WHERE is_positive(review_text) = TRUE
```

→ 의미적으로 "긍정적"인 리뷰를 찾아야 함

→ 정확한 텍스트와 무관하게 "의미"를 이해해야 함

기존 접근법의 한계

1. Text-to-SQL의 한계:

사용자: "이 이미지와 분위기가 비슷한 제품을 찾아줘"

기존 접근 (Text-to-SQL):

1. 사용자의 자연어를 SQL로 변환

→ "분위기가 비슷하다"를 SQL WHERE 절로 표현하기 불가능

2. 부득이 원래 의도와 다른 SQL 생성

→ 사용자 불만족

이유: SQL 자체가 "정확한 일치"에만 적합

→ "의미적 유사성" 표현 불가

2. 벡터 DB의 한계:

벡터 DB (예: Milvus):

```
SELECT * FROM products
WHERE embedding.similarity(query_embedding) > 0.8
LIMIT 10;
```

→ Top-k 유사도 검색은 가능

→ 하지만 "평균 가격"이나 "카테고리별 그룹핑" 같은
관계형 연산과의 결합이 자연스럽게 않음

필요한 쿼리:

```
SELECT category, AVG(price), COUNT(*) as count
FROM products
WHERE embedding.similarity(query_embedding) > 0.8
GROUP BY category
HAVING count > 5;
```

→ 벡터 DB에서는 이런 복합 분석이 어려움 (SQL 미지원)

3. 외부 파이프라인의 복잡성:

현재 실무:

1. SQL DB에서 데이터 추출
2. Python에서 텍스트 전처리
3. Hugging Face에서 임베딩 생성
4. Milvus에 벡터 삽입
5. Milvus에서 검색 실행
6. 결과를 SQL로 다시 분석
7. 시각화

→ 6단계가 필요! 유지보수 난제

9.2 핵심 기여: 시맨틱 SQL 토큰

AnDB는 SQL 문법에 3가지 시맨틱 토큰을 추가하여, "의미 기반 분석"을 SQL 자체에서 직접 표현할 수 있게 한다.

토큰 1: PROMPT (LLM 호출)

개념: WHERE 절 내에서 LLM을 직접 호출하여, 텍스트의 의미를 판단한다.

```
-- 기존 (불가능):
SELECT * FROM reviews
WHERE review_is_positive = TRUE;

-- AnDB (가능):
SELECT * FROM reviews
WHERE PROMPT('이 리뷰가 긍정적인가?', content) = 'positive';
```

동작 방식:

데이터:

review_id	content	PROMPT 결과
1	"정말 좋음"	→ LLM 호출 → "이 리뷰 긍정적?" ← "positive"
2	"별로네"	→ LLM 호출 → "이 리뷰 긍정적?" ← "negative"
3	"괜찮음"	→ LLM 호출 → "이 리뷰 긍정적?" ← "neutral"

결과:

긍정적인 리뷰: 1개, 부정적: 1개, 중립: 1개

실제 사용 예제:

```

-- 질의응답 자동 처리
SELECT customer_id, COUNT(*) as question_count
FROM customer_messages
WHERE PROMPT('이것이 질문인가?', message) = 'yes'
GROUP BY customer_id;

-- 부정적 피드백 자동 탐지
SELECT customer_id, message
FROM reviews
WHERE PROMPT('이 리뷰에 불만이 있는가?', content) = 'yes'
AND PROMPT('긴급한 이슈인가?', content) = 'yes';

-- 지정된 언어의 메시지만 추출
SELECT * FROM messages
WHERE PROMPT('이 메시지가 한국어인가?', text) = 'yes';

```

비용-효율성 문제: - 각 행마다 LLM 호출 → API 비용 증가 - 예: 100만 개 리뷰 분석 시, 100만 번의 LLM 호출 필요 - GPT-4 기준 약 100만원 비용 소요 가능 - → 옵티마이저가 어떤 행에 LLM을 호출할지 선택해야 함 (나중에 설명)

토큰 2: SEM_MATCH (시맨틱 유사도 매칭)

개념: 벡터 유사도 검색을 SQL로 선언적으로 표현한다.

```

-- 기존 (Vector DB):
-- Milvus에서 별도로 벡터 검색 수행

-- AnDB (SQL에서 직접):
SELECT * FROM products
WHERE SEM_MATCH(description, '가볍고 휴대하기 편한 노트북') > 0.8;

```

동작 방식:

입력:

description	SEM_MATCH
"가벼운 울트라북"	embedding →
"15인치 화면"	유사도 계산 → 0.85 ✓
"무거운 데스크탑"	(threshold 0.8 초과)

결과: 첫 번째 상품만 반환

실제 사용 예제:

```

-- 중복 상품 찾기
SELECT a.product_id, b.product_id
FROM products a, products b
WHERE a.product_id < b.product_id
AND SEM_MATCH(a.description, b.description) > 0.85;

-- 유사한 고객 찾기
SELECT u1.user_id, u2.user_id, SEM_MATCH(u1.preference, u2.preference) as similarity
FROM users u1, users u2
WHERE SEM_MATCH(u1.preference, u2.preference) > 0.7
AND u1.user_id < u2.user_id;

-- 관련 뉴스 그룹화
SELECT news_id, SEM_MATCH(title, '인공지능 규제') as relevance
FROM news_articles
WHERE SEM_MATCH(title, '인공지능 규제') > 0.6;

```

토큰 3: SEM_GROUP (시맨틱 그룹핑)

개념: 시맨틱 유사성을 기준으로 행들을 그룹화한다.

```

-- 기존 (불가능):
SELECT * FROM products
GROUP BY semantic_cluster; -- SQL에는 이런 함수 없음

-- AnDB (가능):
SELECT SEM_GROUP(description, 3) as cluster,
       AVG(price) as avg_price,
       COUNT(*) as count
FROM products
GROUP BY cluster;

```

동작 방식:

입력 상품들:

1. "가벼운 울트라북 - 1kg" (카테고리 내부적으로는 "경량 노트북")
2. "얇은 모바일 워크스테이션 - 0.9kg" (역시 "경량 노트북")
3. "게이밍 데스크탑 - 8kg" (다른 카테고리)
4. "12인치 태블릿 - 500g" ("경량 기기")
5. "휴대용 외장 HDD - 300g" ("경량 기기")

SEM_GROUP(..., 3): 3개 클러스터로 그룹화

클러스터 1 (경량 노트북): [1, 2]

클러스터 2 (경량 기기): [4, 5]

클러스터 3 (무거운 장비): [3]

결과:

cluster	avg_price	count	
1	1,500,000	2	(경량 노트북)
2	300,000	2	(경량 기기)
3	2,000,000	1	(무거운 장비)

복잡한 예제:

-- 제품 카테고리를 설명의 의미에 따라 자동으로 재분류

```
SELECT
  SEM_GROUP(description, 10) as semantic_category,
  AVG(price) as avg_price,
  MIN(rating) as min_rating,
  COUNT(*) as product_count
FROM products
GROUP BY semantic_category
HAVING COUNT(*) > 5
ORDER BY avg_price DESC;
```

-- 고객의 선호도를 의미 기반으로 클러스터링

```
SELECT
  SEM_GROUP(preference, 5) as preference_cluster,
  COUNT(*) as customer_count,
  AVG(lifetime_value) as avg_ltv
FROM customers
GROUP BY preference_cluster;
```

9.3 시맨틱 토큰의 통합 활용

예제: 복합 시맨틱 분석

```
-- 긍정적인 리뷰를 가진 상품들을 의미 기반으로 분류
SELECT
  p.product_id,
  p.name,
  SEM_GROUP(p.description, 5) as product_cluster,
  COUNT(*) as positive_review_count,
  AVG(r.rating) as avg_rating
FROM products p
JOIN reviews r ON p.product_id = r.product_id
WHERE PROMPT('이 리뷰가 긍정적인가?', r.content) = 'yes'
AND SEM_MATCH(r.content, '품질이 좋다') > 0.7
GROUP BY p.product_id, p.name, product_cluster
HAVING COUNT(*) > 10
ORDER BY avg_rating DESC;
```

실행 흐름:

1. 모든 리뷰를 PROMPT로 감정 분석
↓ (필터링: 긍정적인 리뷰만)
 2. 남은 리뷰를 SEM_MATCH로 품질 관련 여부 판정
↓ (필터링: '품질 좋음' 언급한 리뷰만)
 3. 상품을 SEM_GROUP으로 의미 기반 클러스터화
↓
 4. 클러스터별로 평균 평점 계산 및 정렬
↓
- 결과: 품질 좋은 제품들을 의미 기반 카테고리로 분류

9.4 비용-정확도 옵티마이저 (Cost-Accuracy Optimizer)

핵심 문제: 비용과 정확도의 트레이드오프

AnDB의 옵티마이저는 기존의 **비용 모델**에 새로운 차원을 추가한다:

기존 옵티마이저:

- I/O 비용만 고려
- 목표: 최소 비용 계획 선택

AnDB 옵티마이저:

- 1. LLM API 호출 비용 (토큰 수 × 가격)
- 2. 추론 정확도 (모델 크기, 프롬프트 품질)
- 3. 실행 시간 (대기 시간 포함)
- 목표: 예산 내에서 최대 정확도 계획 선택

예제: 비용 계산

```
SELECT * FROM reviews  
WHERE PROMPT('이 리뷰가 긍정적인가?', content) = 'yes';
```

선택지들:

모델	평균 토큰	가격/1K토큰	정확도	총 비용 (100만 리뷰)	총 정확도
GPT-3.5	50	\$0.5	82%	\$25,000	82%
GPT-4	50	\$15	95%	\$750,000	95%
Llama-2	50	\$1	78%	\$50,000	78%
로컬 모델	50	\$0	70%	\$0	70%

의사결정:

시나리오 1: 예산 제약 없음 (정확도 최우선)

→ GPT-4 선택 (95% 정확도)

시나리오 2: 예산 \$100,000 이내

→ GPT-4 (750만원 초과) 불가능

→ GPT-3.5 (25,000) 또는 Llama-2 (50,000) 중 선택

→ GPT-3.5 선택 (82% 정확도, 가성비 우수)

시나리오 3: 예산 \$5,000 이내

→ 로컬 모델만 가능 (70% 정확도)

시나리오 4: 필터링으로 선택적 적용

→ 전체 100만 리뷰에 GPT-4 적용 불가능

→ 먼저 저비용 로컬 모델로 "명백히 긍정적"인 것들만 필터링

→ 모호한 것들만 GPT-4로 정교한 분석

→ 전체 비용 크게 감소 & 정확도 유지

옵티마이저의 선택 메커니즘

쿼리: 100만 리뷰 감정 분석, 예산 \$100,000

옵티마이저의 고민:

옵션 1: 모든 리뷰에 GPT-3.5 (비용: \$25,000, 정확도: 82%)

└ 문제: "명백히 긍정" "명백히 부정"을 선불리 판단

옵션 2: 2단계 필터링

└ 1단계: 로컬 모델로 "명백한" 사례들 필터링 (비용: \$0)

└ 결과: 100만 중 50만 개로 축소 (명백한 긍정/부정)

└ 2단계: GPT-3.5로 남은 50만 개 분석 (비용: \$12,500)

└ 총 비용: \$12,500, 정확도: ~90% (1단계 로컬의 확신도 반영)

옵션 3: 비율 기반 샘플링

└ 예산 배분: 저비용 모델 50%, 고비용 모델 50%

└ 모든 리뷰를 저비용 모델로 빠르게 스캔

└ 그중 일부를 고비용 모델로 정밀 검증

└ 총 비용: \$100,000, 정확도: ~92%

옵티마이저 선택: 옵션 2

→ 비용은 최소, 정확도는 충분하고, 자신감도 높음

9.5 본 논문과의 관계

근본적인 설계 철학의 차이

측면	Exqutor	AnDB
설계 철학	"SQL을 확장하자"	"SQL을 재설계하자"
벡터 검색	ORDER BY + LIMIT로 표현	SEM_MATCH로 직접 표현
LLM 연동	지원 안 함	PROMPT로 직접 호출
그룹핑	GROUP BY + 필터링	SEM_GROUP으로 의미기반
대상 DB	범용 벡터 DB (pgvector, VBASE)	AI-native DB

발전 시나리오: AnDB가 표준이 되면?

현재 (Exqutor의 세상):

- SQL이 기본 언어
- 벡터 검색은 SELECT 조건으로 표현
- LLM은 애플리케이션에서 호출

미래 (AnDB의 세상):

- SQL이 기본 언어이지만 시맨틱 토큰 포함
- 벡터 검색은 SEM_MATCH로 직접 표현
- LLM은 SQL에서 PROMPT로 호출
- 옵티마이저가 비용-정확도 균형

이 미래에서 Exqutor의 역할:

1. SEM_MATCH의 카디널리티 추정
→ "이 쿼리 결과가 몇 개일까?"를 정확히 예측
2. SEM_GROUP의 클러스터 크기 추정
→ "각 클러스터에 몇 개 행이 들어갈까?"
3. PROMPT의 선택도 추정
→ "이 프롬프트 필터가 몇 %를 제거할까?"

결론: AnDB 같은 시스템이 발전할수록,
Exqutor 스타일의 정확한 카디널리티 추정이
****더욱 필수적****이 된다!

9.6 비용-정확도 트레이드오프의 깊이 있는 분석

PROMPT 토큰의 비결정성 문제

문제: LLM의 응답이 항상 일관적이지 않음

같은 프롬프트, 같은 텍스트도 LLM에 따라 다른 답:

텍스트: "가격이 좀 비싸네요"

GPT-4: "negative" (불평 감지)

Llama-2: "neutral" (서술적 관찰로 해석)

로컬 모델: "positive" (어떤 모델인지에 따라)

결과:

- 같은 데이터, 같은 SQL
- 다른 LLM 선택 → 다른 결과 반환
- 옵티마이저가 "얼마나 많은 행을 필터링할지" 예측 불가

이것이 왜 중요한가?

```
SELECT * FROM reviews
WHERE PROMPT('이것이 불평인가?', content) = 'yes'
AND price > 100000;
```

카디널리티 추정 계산:

Exqutor 스타일 정확 추정이 있다면:

PROMPT 결과: 전체의 30% = 300,000개
price > 100000: 전체의 20% = 200,000개
결합 (AND): $300,000 * 200,000 / 1,000,000 = 60,000$ 개

하지만 LLM의 비결정성 때문에:

실제 실행 결과:
- GPT-4: 35만개 (엄격한 정의)
- Llama: 25만개 (너그러운 정의)
- 로컬: 15만개 (제한적 정의)

±50%의 오차 발생!

9.7 실제 구현 가능성에 대한 의문점

논문은 "데모"로 분류되어 있으며, 다음과 같은 실무적 문제들이 명확하게 해결되지 않았다:

1. **대규모 LLM 호출의 실시간성:**
2. 100만 개 행을 쿼리할 때, LLM API 호출이 병목이 된다
3. "텍스트당 100ms"라고 해도 $100만 \times 100ms = 100,000초 = 28시간$ 소요
4. 배치 처리만 가능할 가능성 높음
5. **API 가용성과 레이턴시:**
6. 동시에 수만 개의 API 호출 시 rate limiting 발생
7. API 응답 시간이 불안정하면 쿼리 총 시간도 불안정
8. **비용 폭증:**
9. 대량의 데이터에 대해 PROMPT를 사용하면 비용이 기하급수적 증가
10. 예: $1000만 \text{ 행} \times \text{GPT-4} = \$7.5M$ (한 번의 쿼리 비용!)
11. **결정성(Determinism) 보장 불가:**
12. 같은 쿼리를 여러 번 실행하면 결과가 달라질 수 있음

13. 트랜잭션의 ACID 특성 위반

14. 메모리 효율성:

15. 각 행의 PROMPT 결과를 캐싱해야 비용 절감 가능

16. 캐시 관리와 갱신 전략 불명확

추가 제기 문제

1. **SEM_MATCH의 정확도와 차원성**: 시맨틱 매칭이 고차원(1000+) 벡터에서 어떻게 동작하는지, 특히 "0.8이 의미 있는 임계값인가?"에 대한 분석 부족
2. **SEM_GROUP의 클러스터 품질**: 3개나 5개 클러스터로 나누는 기준이 무엇인가? 데이터마다 다를 텐데, 자동으로 최적의 클러스터 수를 결정하는 방법이 없음
3. **트랜잭션과 일관성**: AnDB에서 PROMPT 결과가 시간에 따라 변할 수 있다면, 트랜잭션 일관성을 어떻게 보장할 것인가?
4. **공정성(Fairness) 문제**: LLM의 편향이 쿼리 결과에 전파될 수 있다. 예를 들어, 특정 집단을 차별하는 LLM을 사용하면, DB 쿼리 결과도 차별적이 됨
5. **감사(Audit) 추적**: "왜 이 행이 결과에 포함되었나?"를 설명하기 어려움 (LLM의 "블랙박스" 특성)